

Flight Inspectors

Process Book · Milestone 3

"How safe do you think flying really is?"

5,078

Crashes recorded

93,197

Civilian fatalities

116

Years of aviation data

TEAM / FLIGHT INSPECTORS

R

Roméo Maignal

SCIPER 360568 · Website architecture · Viz 1 & 2

N

Nicolas Karmolinski

SCIPER 316655 · Viz 5 (Poisson) · Data analysis

J

Jakub Kielar

SCIPER 423372 · Viz 3 & 4 · Globe · Heatmap · Treemap

Understanding the real risks of air travel

The Problem

No matter how statistically safe modern air travel is, the feeling of insecurity when boarding a plane persists. Crash events dominate media coverage, distorting public perception of actual risk. We wanted to address this with data.

Our Approach

We built an interactive web platform that lets users explore aviation safety across six distinct visualizations—spanning manufacturer records, airline casualty trends, crash geography, and a Poisson-based predictive model for personalised risk assessment.

The Dataset

Primary: Kaggle 'Plane Crashes Dataset' (1908–2024) by Luiscé Francisco. Supplemented by OpenFlights airport, route and airline tables. Processing focused on isolating commercial flights and geo-locating routes for the globe visualization.

Research questions

- **Which manufacturers** have the safest long-term incident record?
- **How do casualty rates** per airline change decade by decade?
- **Where in the world** do crashes cluster, and on which routes?
- **Can Poisson statistics** model the probability of an incident on your flight?

Tech stack

React + TypeScript

UI & component model

Vite

Build tool & dev server

D3.js

All data visualizations

TailwindCSS

Styling framework

GitHub Pages

Hosting & CI/CD

Python

Data preprocessing scripts

Development workflow

- main**
Website foundation, Viz 1 & 2, UI polish
- world_maps**
Viz 3 (Heatmap + Treemap) and Viz 4 (Interactive Globe)
- poisson**
Viz 5 – Poisson distribution model

All three branches were merged into main and deployed automatically to GitHub Pages via a CI workflow triggered on every push.

From dataset to design

Milestone 1 established our dataset, research objectives and related work. We settled on the Kaggle 'Plane Crashes' dataset (1908–2024) complemented by OpenFlights data for geo-coordinates. The core vision: six interactive visualizations revealing different facets of aviation safety, culminating in a Poisson model that lets users estimate the risk of their own flight.

Six planned visualizations

Viz 1

Manufacturer safety record

Bar chart comparing incident & fatality counts per aircraft maker across a selectable year range.

Viz 2

Airline casualty bubble chart

Animated bubble chart where bubble size encodes cumulative casualties, driven by a time slider.

Viz 3

Crash-location heatmap + treemap

World choropleth showing crash density by country; treemap below for proportional breakdown.

Viz 4

Interactive globe with crash routes

Rotatable 3D globe plotting each route with arc width proportional to fatalities.

Viz 5

Poisson distribution model

Statistical tool: the user selects origin, destination, airline and manufacturer to obtain a Poisson PMF curve estimating incident probability.

Viz 6

Directed chord diagram

Originally planned to show flight-route flows between world regions. Deprioritised in favour of completing the globe and Poisson model.

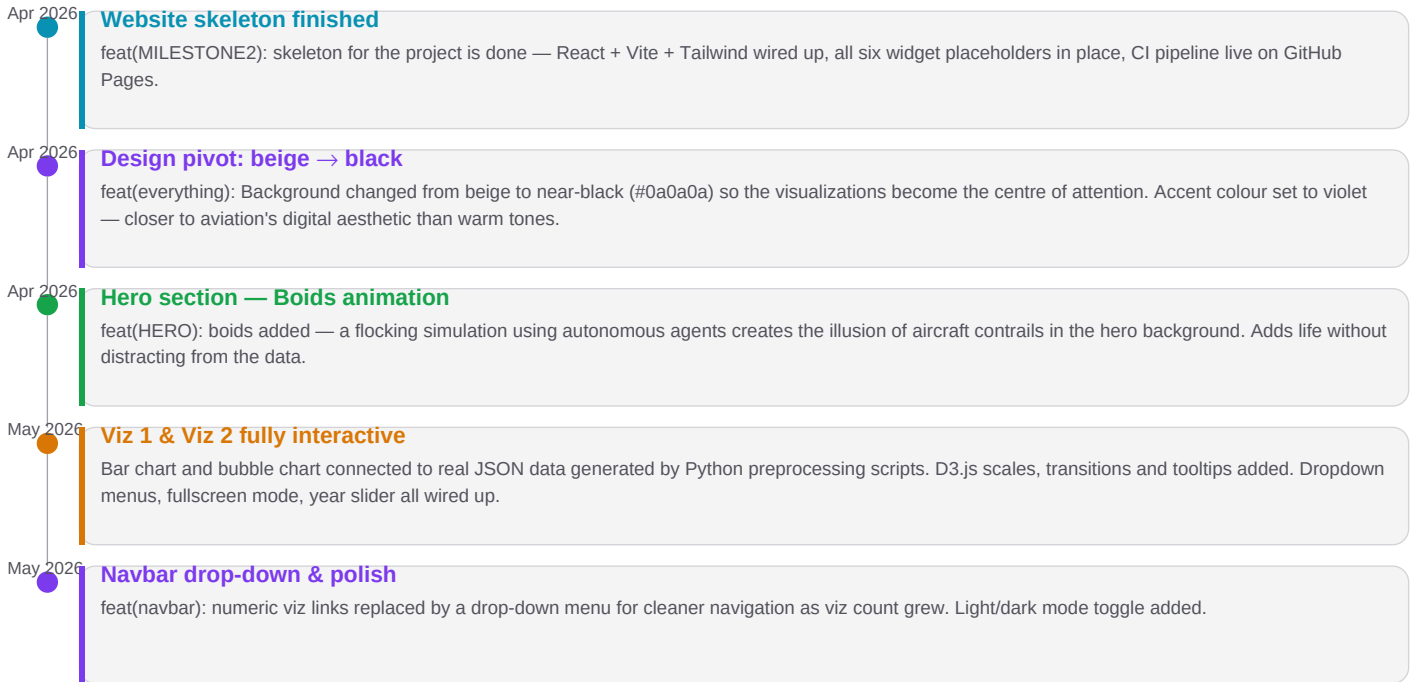
Key inspirations

- **FlightRadar24** Real-time flight tracking — inspired our route-arc approach on the globe.
- **FlightConnections** Interactive route map — showed us how to make the globe explorable.
- **OpenFlights Map** Static world flight map — we wanted to add crash data and interactivity.
- **Kaggle analyses** Prior work on similar datasets gave us benchmarks for our own EDA.

Building the platform

Milestone 2 turned the design sketches into a running website. The core skeleton—navigation bar, hero section, and widget scaffolding for all six visualizations—was delivered in a single PR on main. The following weeks brought major visual refinements and the first two fully interactive visualizations.

Key milestones on main



Design decisions & trade-offs

Dark theme

Initially the site had a warm beige background. After building the first visualizations it became clear that colourful charts need a neutral canvas. Near-black (#0a0a0a) lets the D3 colour palettes pop without competing hues.

Widget + fullscreen pattern

Every visualization lives in a self-contained 'widget' card. A single useFullscreen hook grants every viz a consistent fullscreen button, avoiding duplicated code and ensuring a unified UX.

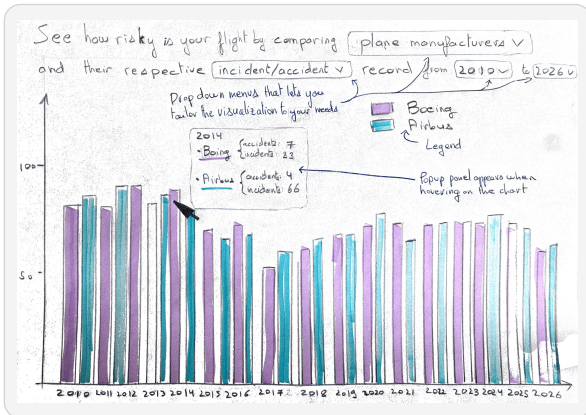
Navbar backdrop blur issue

On the local Vite server the translucent navbar blur rendered correctly. GitHub Pages strips certain CSS features. We resolved this by setting a solid dark background for the deployed version while keeping blur locally.

Manufacturer & airline records

Viz 1 — Manufacturer safety record (bar chart)

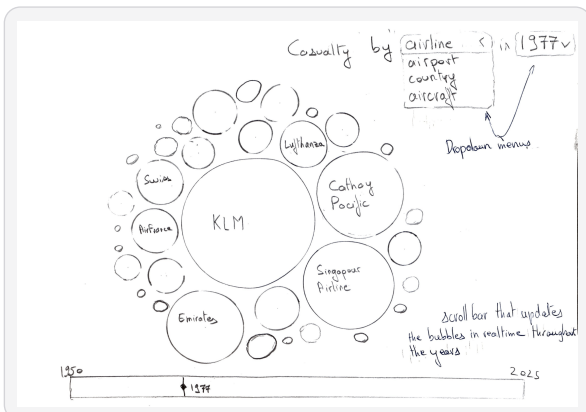
Users select up to four aircraft manufacturers and a metric (incidents or fatalities) to compare their yearly records on a grouped bar chart. Dropdown menus and an optional year range let visitors tailor the view. Hovering a bar triggers a D3 tooltip with exact figures.



Milestone 2 sketch → implemented as-is with minor layout refinements

Viz 2 — Airline casualty bubble chart

A bubble-chart showing cumulative casualties (or incident counts) per airline as of a given year. A time slider animates the layout from 1919 to 2024. Bubbles are colour-coded, repel each other with a D3 force simulation, and a glow effect fires on hover to highlight the selected airline.



Milestone 2 sketch — bubble positions evolved via D3 force simulation

Challenge

Aligning grouped bars cleanly when fewer than four manufacturers are selected. Solved with D3 scaleBand for the outer group and a nested scaleBand for individual bars.

Data pipeline

Python script (data/viz1/script.py) reads crashes_cleaned.csv, groups by manufacturer key and year, then writes public/data/crashes_by_manufacturer.json consumed by the React component.

Colour palette

Each manufacturer gets a fixed accent colour from a hand-picked set of 15 so repeated views are consistent. Boeing → violet, Airbus → cyan.

Challenge

Force simulation settling time vs. slider responsiveness. We pre-warm the simulation and debounce slider events so bubbles glide rather than jump.

Year label

The currently selected year is rendered directly on the slider thumb so the user never loses context while dragging — small but impactful UX detail.

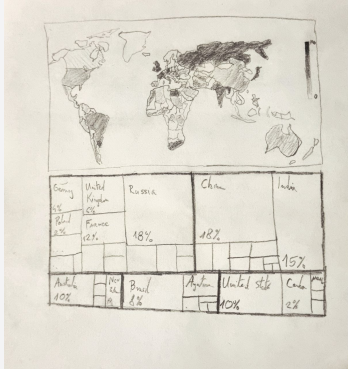
Pivot from sketch

The original sketch showed a static layout. The live version uses a forceSimulation so bubbles never overlap, which required adding rest-position anchors to keep the cloud centred after each tick.

Geography of aviation crashes

Viz 3 — Crash heatmap (choropleth) + treemap

A dual-panel widget: the upper panel is a Mercator world choropleth where each country is shaded by incident or fatality count over a selectable year range. The lower panel is a treemap giving proportional area to each country. Hovering either panel cross-highlights the corresponding entry in the other.



Milestone 2 sketch — heatmap + treemap layout preserved in final implementation

TopoJSON

Country geometries come from the world-atlas package (110m resolution). `topojson-client.feature()` converts topology to GeoJSON for D3 `geoPath`.

Mercator crop

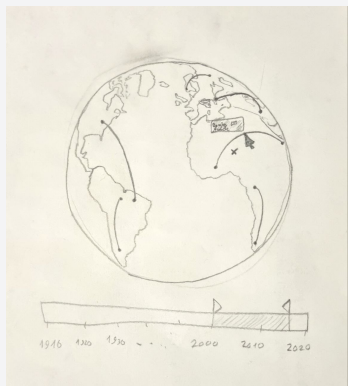
Antarctica wastes space. We wrote a custom `makeHeatProjection()` that clips to lat $[-55, 84]$, fitting the strip precisely into the widget.

Continent colours

Countries are coloured by continent (Africa=red, Asia=yellow, Europe=cyan...) in the treemap so regional patterns emerge at a glance.

Viz 4 — Interactive globe with crash routes

A rotatable 3D globe (D3 `geoOrthographic`) plots each crash route as an arc. Arc thickness scales with fatalities. The globe auto-rotates when idle; drag pauses it. A year-range dual slider filters routes. Clicking a route pins a tooltip showing operator, aircraft type, origin, destination and fatality count.



Milestone 2 sketch — implemented with D3 `geoOrthographic` & drag interaction

Auto-rotate

A `requestAnimationFrame` loop rotates the globe at 0.0008 rad/tick. Any pointer event resets a 2.5 s idle timer, keeping the globe still while the user explores before gently resuming.

Route data

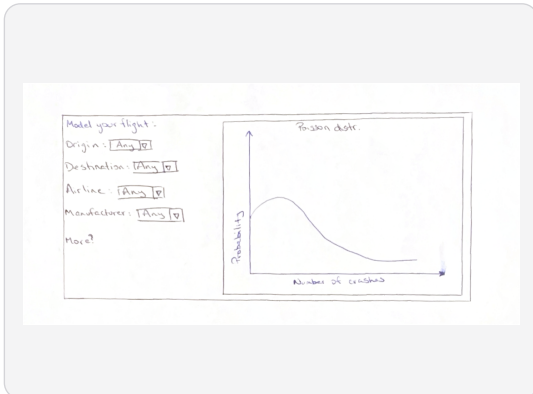
Python (`data/viz3/script.py` + `viz4/script.py`) cross-references crash records with OpenFlights to resolve origin/destination city coordinates, writing `crashed_routes.json` and `crashes_by_country.json`.

Design change

The original sketch showed a flat 2D map. We switched to a 3D orthographic globe to allow free exploration of all routes globally and make the widget more visually engaging and unique.

Statistical risk modelling

Visualization 5 is the most analytically ambitious part of the project. Rather than simply displaying historical data, it uses a Poisson model to give users a personalised probability estimate: given a choice of manufacturer, airline, origin and destination, what is the probability of 0, 1, 2 ... incidents occurring in the next year?



Milestone 2 sketch — 'Model your flight' with Poisson curve

Poisson model — theory

We estimate λ (average incidents/year) empirically from the filtered crash records for a user-defined combination of manufacturer, airline, origin country and destination country over a selected decade. The PMF $P(X=k) = e^{-\lambda} \cdot \lambda^k / k!$ is then plotted as a bar chart up to $k_{\max} \approx \lambda + 4\sqrt{\lambda}$. $\lambda=0$ collapses to a degenerate spike at $k=0$.

Implementation details

Filters & combinations

The component receives a pre-aggregated JSON (data/viz5/script.py) containing incident counts grouped by (manufacturer × airline × origin × destination × year). Four cascading dropdowns let users narrow to any combination; the λ is recomputed client-side on every change without a network round-trip.

Log-factorial cache

Computing $k!$ for large k is numerically unstable. We accumulate log-factorials in a cache array (`_logFactCache`) and use the identity $\log P(X=k) = -\lambda + k \cdot \log(\lambda) - \log(k!)$ to stay in floating-point range.

Year-range presets

Six preset decade buttons (All time, 2000–2024, 1990–1999 ...) let users quickly compare how λ has changed over time for the same combination—visually showing whether a given airline has improved its safety record.

UX decision

The chart updates instantly on every dropdown change (no 'Submit' button). This encourages exploratory play and makes the statistical feedback feel immediate. We debated adding a spinner for perceived loading; decided the computation is fast enough that it would feel patronising.

Branch strategy

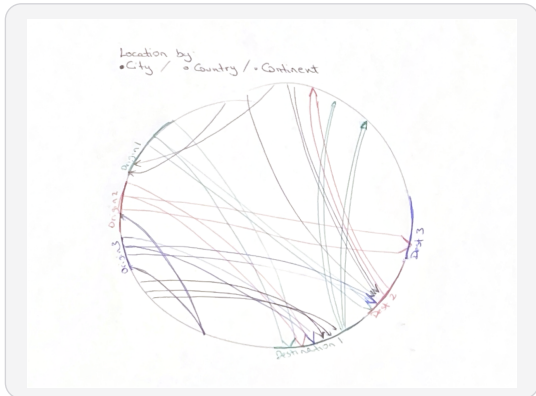
Viz 5 was developed entirely on the poisson branch to avoid blocking main with an incomplete widget. The branch diverged from main at commit c9395b9 and adds only Viz5.tsx, Viz5.css and the data/viz5 preprocessing script.

■ Key challenge

Sparse combinations (e.g. a niche regional airline + obscure manufacturer) yield $\lambda \approx 0$, producing a degenerate distribution that looks alarming to non-statisticians. We added a contextual note explaining that $\lambda=0$ means 'no recorded incidents for this combination' — not 'guaranteed crash-free' — to preserve statistical honesty.

Directed chord diagram

Visualization 6 maps crash flows between world regions as a directed chord diagram. Each arc connects an origin region to a destination region, with width proportional to the number of crashes on that route. The layout makes it easy to spot which regions are most linked by fatal incidents.



Milestone 2 sketch - directed chord diagram with regional crash flows

Design overview

World regions are placed as segments on a circle. Directed chords connect origin to destination, thickness encoding crash-flow volume. D3 chord layout handles angular positioning from the flow matrix. Colors follow the continent scheme used in Viz 3.

Implementation details

Node count

Finding the right number of nodes took some trial and error. Too many made the chart unreadable, too few lost geographic detail. Grouping by continent gave a clean layout while keeping the flows meaningful.

Self-connecting arcs

Crashes where origin and destination fall in the same region produced self-connecting arcs that cluttered the center of the diagram. We removed them since intra-region flows are less interesting to show.

Data pipeline

Python script cross-references crash records with OpenFlights to assign each crash to an origin and destination region, then builds the flow matrix fed into the D3 chord layout.

Arc direction

The thicker end of each arc marks the origin region, the thinner end the destination. Region label colors carry over to the arcs so flows are easy to trace across the diagram.

GitHub Pages

The SVG rendering of the chord diagram looked fine locally but some arc gradients were dropped on GitHub Pages. Switching from CSS gradients to plain SVG fill colors kept the chart consistent across environments.

Who built what

R

Roméo Maignal

SCIPER 360568

■ Website skeleton & CI/CD

Set up the React + Vite + TailwindCSS project and the GitHub Pages pipeline. Getting everything to render properly on Pages was tricky: Vite's base path had to be configured manually or all asset URLs broke on deploy while working fine locally.

■ Visualization 1 - Manufacturer bar chart

Grouped bar chart with D3 scaleBand, dropdown filters and animated transitions. GitHub Pages kept serving cached JSON so we added cache-busting to the data fetch to make sure the latest file was always loaded.

■ Visualization 2 - Airline bubble chart

D3 force-simulation bubble chart with year slider and glow-on-hover. The navbar backdrop-blur worked on the dev server but broke on GitHub Pages, so we used a solid background for the deployed version.

J

Jakub Kielar

SCIPER 423372

■ Visualization 3 - Heatmap + Treemap

Mercator choropleth with custom latitude clipping linked to a treemap with cross-highlighting. The TopoJSON bundle was large enough to cause a blank map on GitHub Pages, fixed by lazy-loading the file and showing a loading state in the meantime.

■ Visualization 4 - Interactive Globe

3D orthographic globe with arc drawing, auto-rotate and drag. On GitHub Pages the globe sometimes started spinning before assets finished loading, showing blank frames. Waiting for all files before starting the animation fixed it.

N

Nicolas Karmolinski

SCIPER 316655

■ Visualization 5 - Poisson distribution model

Dropped the airline filter since too many carriers are discontinued, making results hard to read. Used decade presets instead of single years for more useful estimates. Cached log-factorials to avoid floating-point overflow for large k values.

■ Visualization 6 - Directed chord diagram

Finding the right number of nodes took trial and error to keep the chart readable without losing too much detail. Removed self-connecting arcs as they looked cluttered. Arc width is proportional to crash-flow volume between regions.

Contribution split (estimated):

■ **Roméo Maignal:** ≈ 34 % · skeleton + Viz 1 & 2

■ **Jakub Kielar:** ≈ 33 % · Viz 3 & 4

■ **Nicolas Karmolinski:** ≈ 33 % · Viz 5 & 6